

STT Benchmarking & Optimization Report

1. Executive Results Summary

This report details the execution and results of the STT Benchmarking Framework. The evaluation measures the Word Error Rate (WER) to assess accuracy and captures latency percentiles (P50, P95) to measure responsiveness. Note: Current results reflect initial simulated baselines to validate the pipeline architecture.

2. Individual Model Scorecards & Metrics

Model: SARVAM_SAARAS

- Average WER: 100.00%
- Average CER: 88.14%
- P50 Latency (Median Response): 695.91 ms
- P95 Latency (Tail Response): 712.01 ms

3. Global Benchmarking Methodologies for Data Harnessing

To effectively harness data for STT evaluation, we employ three distinct benchmarking methodologies:

A. Controlled Benchmark (Scientific Baseline):

Uses pristine, clean audio with exact reference texts. The goal is to measure the pure acoustic and language modeling capabilities of the STT without external interference. This tests baseline WER and ideal-case latency.

B. Real-World Production Benchmark (Robustness):

Injects production-grade artifacts such as background noise, telephony 8kHz downsampling, network-induced packet loss, and heavy Indian accents. It answers 'How does this perform in our actual call center environment?'

C. Adversarial/Failure Benchmark (Edge-Case Hunting):

Focuses strictly on known failure modes: code-mixing (e.g., Hindi-English switching), heavy acronym usage, financial numbers ('INR 25,000 EMI'), and rapid speech. It is used to actively break the models to discover critical blind spots.

4. Architectural Optimization Strategies

A. Ideas to Improve Latency:

- Edge VAD (Voice Activity Detection): Trim silence on the client/edge device before sending audio to the

STT Benchmarking & Optimization Report

cloud API. Less audio payload equals faster transit and processing time.

- Streaming Chunk Optimization: Adjust chunk sizes. A 50ms chunk might cause network overhead, while a 500ms chunk delays the first partial response. Finding the optimal chunk size (e.g., 200ms) dramatically improves perceived latency.
- Persistent Connections (WebSockets/gRPC): Avoid HTTP handshake overheads for every request by maintaining a warm connection pool.

B. Ideas to Improve Core Metrics (Accuracy):

- Custom Vocabulary / Boosting: Pass context (e.g., product names, competitor names) as hints in the API request if the provider supports it.
- Semantic Error Rate (LLM-as-a-Judge): Standard WER penalizes '22' vs 'twenty two'. Using an LLM to evaluate semantic equivalence provides a truer reflection of business value than raw WER.

C. Ideas to Improve Hashing/Storage Cost:

- Hash Caching: Instead of recalculating SHA-256 for identical audio files on every run, store hashes in a local SQLite index/cache.
- Payload Compression: Compress audio using FLAC or Opus before hashing and storage. This reduces storage cost for the evidence database while maintaining mathematical integrity of the original payload.